

HOW GOOD IS YOUR HAND, REALLY??

Introduction

Growing up playing Chinese poker games recreationally with family, two phrases I often heard were “man, my hand is so bad!” or “OMG, my hand is so good!!” Last quarter, I started learning how to play mahjong in order to play with my lion dance team at Stanford. Similar to poker, mahjong also has randomly drawn starting hands that are based on luck. Although mahjong is a very complicated game, it’s also really fun and interesting, so I wanted to do a project related to it—especially since a lot of it is very much based on luck and probability. My project attempts to quantify how “good” a starting mahjong hand is based on its win and point (Faan) potential, so that I can evaluate how good a hand *actually* is.

Background

In Hong Kong mahjong, there are 144 tiles that are shuffled, and at the start of each game, each player draws 13 tiles. Depending on certain conditions (such as drawing certain special tiles), they may then draw additional tiles to be a part of the starting hand. The goal of mahjong is to be the first person to complete your hand, which requires forming four sets of three tiles along with a pair. Tiles can form sets of three if they are all the same tile, or if they are all from the same suite with consecutive numbers. In Hong Kong mahjong, a winning hand has a point, or Faan, value, which determines the amount of money that is owed to the winner. Points are based upon multiple conditions of the winning hand. Importantly, you usually need a minimum of three Faan in order to win the game, in addition to having to complete your hand. This means that in order for a starting hand to be good, it must satisfy the potential for both requirements.

Dataset

I created my own dataset by making a mahjong “deck” in python. Since a mahjong deck has 144 tiles and a starting hand has 13 tiles, there are in theory $144 \text{ choose } 13 = 1.05213437e18$ or about 1 million trillion different starting hands. The actual number of different hands is less since some of the tiles are identical, but regardless, the amount is too large to evaluate each unique hand. Instead, I took 100,000 random samples of size 13 from the mahjong deck to form my dataset. Each sample represents a different starting hand.

It is difficult to objectively quantify the value of a starting hand. Counting existing Faan is not a good metric since it’s meant to be applied to winning hands, and will usually be 0 for a starting hand. It’s also difficult to measure the exact “distance” of a hand from a winning state, since for a randomly drawn starting hand, there could be multiple strategies that lead to a win. Thus, I created a scoring methodology based on multiple factors such as potential win and Faan value

(see appendix). Naturally, this also results in “rarer” combinations of tiles yielding higher point values, since the Faan system is also designed to reflect that. Other features of my dataset included keeping track of the number of pairs, flowers, sets of three, tiles in a row, and more for each hand. I then calculated the score for each sample hand that I drew and used it to create the distribution for starting hand efficiency. This was done by taking the counts of hands that had each score and dividing by 100,000 (the number of samples) in order to create a PMF estimation. The data has a mean score of 9.33424 and a variance of 8.745191074310743. The distribution of scores among the dataset is as shown in Figure 1 below.

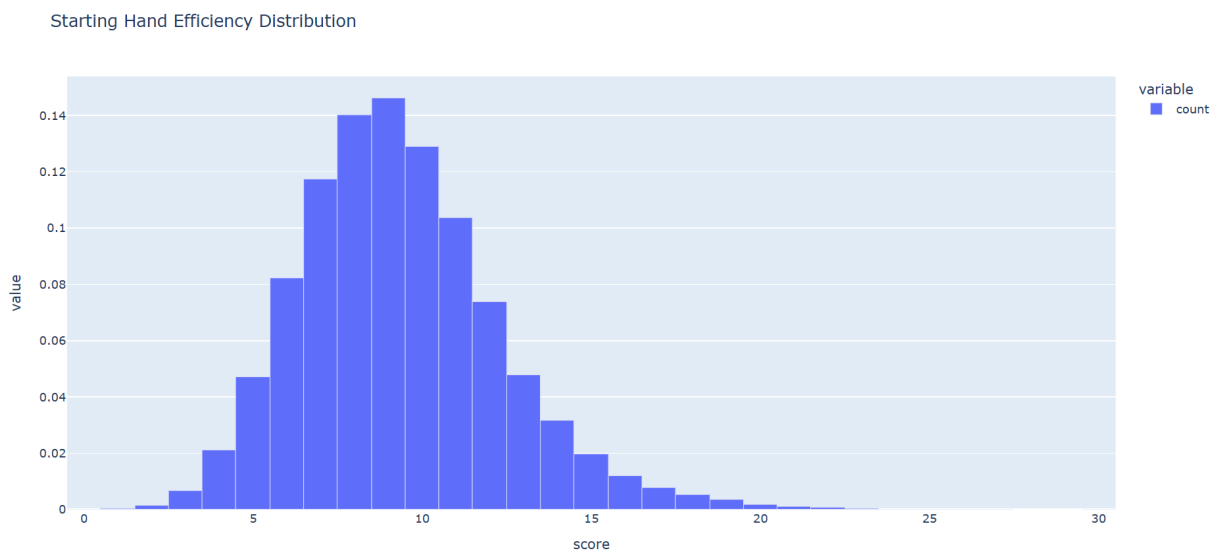


Figure 1: The distribution of scores for starting hand efficiency.

At a glance, the distribution’s shape appears close to a normal distribution that is slightly right skewed. This occurs because of the Central Limit Theorem. Each component of the scoring system (ex. pairs, sequences, etc.) is close-to independently and identically distributed (within a hand, tiles are drawn without replacement, but there are so many tiles that the difference becomes negligible). This means that the sum of these many random variables of how many pairs the hand has, how many sequences the hand has, etc. causes the distribution to approach a Gaussian over many samples. The reason that this particular distribution is still right skewed is because the scoring system has some features, including rare high-value combinations, which cause certain hands to have a much higher score than others and induce a heavier right tail. These “outliers” cause the distribution to not truly follow a Normal distribution.

Method and Modeling

I trained two linear regression models on the same data, but with different input features for each model, using sklearn’s LinearRegression. A regression model is more suitable for this project

than a classification model since it can make more specific predictions of quantified measures for how good the hand is, rather than just saying it is “good” or “bad.” I also thought about using KNeighborsRegressor, but that model runs into the problem of the curse of dimensionality, especially for my second training set, which has 145 parameters.

For my training set, I simulated drawing 100,000 starting hands, and for my testing set, I drew another 25,000 samples, which is an 80/20 train-test split. I then trained each linear regression model on the train set before testing it on the test set.

My first model was trained on the following metrics I measured for each hand: the number of flowers, pairs, pong (sets of three), kong (sets of four), tiles in a row, and suites missing from the hand. My second model was trained on data that had a boolean column for each tile in the deck, representing whether a hand contained each specific tile. It also included the number of flowers. I wanted to see whether the model could predict the score based off of just the tiles alone, without the metrics that I calculated and provided to the first model.

Results and Conclusion

I evaluated both models using the root mean squared error of the test predictions compared to the actual scores of the test set. The first model had a test RMSE of 0.7677223217794757, which means that on average, the score predicted by model 1 was around 0.77 away from the actual score of the hand. This is actually an extremely accurate model, especially for a distribution with variance ~ 8.75 . For model 2, the test RMSE was 2.837560117644207, meaning it was on average about 2.84 away from the actual score of the hand. Thus, model 1 performed better than model 2. This makes sense because a lot of the scoring system I made is almost directly based on the metrics in model 1's train set, whereas a linear model 2 trained on just the tile data has difficulty capturing patterns of the hand such as pairs and sequences that are integral to the scoring. Regardless, it was cool to see that model 2 was also not too far off using only data about which tiles were in the hand.

Discussion

Some of the limitations of my project include the fact that mahjong is a much more complicated game than is represented by my simplified scoring system. My scoring system isn't perfectly faithful to how the game is actually played; thus, there are many possible starting hands whose efficiency isn't accurately represented by my methodology.

If I had more time, I would actually simulate the games being played out in order to more accurately measure which starting hands end up doing well. For example, I could estimate the expected number of draws required to reach “one away from winning,” which would be a more objective measure for how “good” a starting hand actually is.

Appendices

1. The following describes my scoring system:
 - Flowers
 - +1 for no flowers
 - +1 for a flower that matches the player's current direction
 - +1 for a flower that matches the direction the wind is blowing
 - In practice, each of these conditions is +1 Faan
 - Pairs
 - +2 for a pair of suite tiles
 - +3 for a pair of honor tiles
 - +1 if it matches the player's current direction
 - +1 if it matches the direction the wind is blowing
 - In practice, a pair allows you to "pong" (finish the set of three) from the discard pile
 - Pong (set of three)
 - +3 for a pong of suite tiles
 - +4 for a pong of honor tiles
 - +1 if it matches the player's current direction
 - +1 if it matches the direction the wind is blowing
 - In practice, a winning hand that is "all pong" (i.e. no sequences) is +3 Faan
 - Pong of direction tiles that match the player's or the wind's direction is +1 Faan
 - Kong (set of four)
 - +4 for a pong of suite tiles
 - +5 for a pong of honor tiles
 - +1 if it matches the player's current direction
 - +1 if it matches the direction the wind is blowing
 - In practice, a Kong is the same as a Pong, except +1 Faan
 - Chi (sequences)
 - +1 for tiles of the same suite with consecutive numbers
 - +1 for tiles of the same suite with one number in between (ex. 5 and 7)
 - Missing suites
 - +4 per suite that is missing from your hand
 - In practice, a winning hand that is all of the same suite with honor tiles is +3 Faan. A winning hand that is all of the same suite with no honor tiles is +7 Faan.
2. LLM use (tl;dr: not a lot):
 - a. I knew I wanted to focus on expected value of hands/tiles in mahjong. ChatGPT helped brainstorm to narrow the scope and solidify the concept of the project.
 - b. Claude gave me some feedback on extra things I could include in the write-up, but did not actually write anything.